



← Back to student guides

 Docker

Containers

3 levels

133 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.



Part one of three. A twenty-five-minute review of the **Beginner Detailed track**, then the questions interviewers ask. Mid-level reviews this plus its own material; Senior reviews all three.

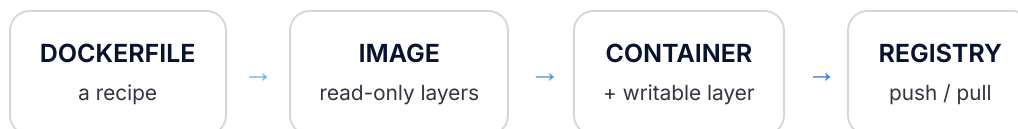




Docker packages an application with its runtime, libraries, and system dependencies into an immutable image. A container is a running instance of that image: a process on the host, isolated by kernel namespaces and cgroups rather than a hypervisor. Because the image carries its own environment, the artifact that passed CI is the artifact that runs in production.

Then the line that shows you have used it: "a container lives as long as its main process, and anything not in a volume disappears when the container is removed."

The model



Term	Say this
Image	A read-only stack of filesystem layers plus metadata
Layer	The filesystem diff produced by one Dockerfile instruction
Container	A running instance: the image's layers plus a thin writable layer
Volume	Docker-managed storage that outlives the container
Bind mount	A host directory mounted into a container
Registry	Where images are stored and shared: Docker Hub, GHCR, ECR
Tag	A mutable human label on an image, <code>myapp:1.2.0</code>
Build context	The directory sent to the daemon at build time
Dockerfile	The recipe that produces an image



Daemon

The background service that does the work, the CLI only talks to it

Container versus VM

✓ Container

- Shares the host kernel
- Milliseconds to start, tens of MB
- Isolated by namespaces and cgroups
- A process with a restricted view

✗ Virtual machine

- Ships its own guest kernel and OS
- Tens of seconds, gigabytes
- Isolated by a hypervisor (stronger)
- A simulated computer

The follow-up: "so is a container less secure?" Yes. Shared-kernel isolation is weaker than a hypervisor, and a kernel vulnerability is an escape path. Drop privileges, and keep untrusted code out of a plain container.

One more fact: containers are a Linux kernel feature, so Docker Desktop runs a small Linux VM on macOS and Windows. Bind-mount file I/O is slower there.

Image versus container

	Image	Container
Is	Read-only template of stacked layers	A running instance
Analogy	A class · an installer	An object · the running program
Mutability	Immutable	Thin writable layer, lost on <code>rm</code>
Count	One	Many from the same image



built by

docker build

docker run

Three containers from one image share its read-only layers on disk, and anything you write outside a volume dies with `docker rm`.

The commands to have ready

BASH

Copy

```
docker run -d -p 8080:80 --name web nginx # detached, port-mapped, named
docker ps -a # all containers; exit codes live here
docker logs -f --tail 50 web # follow output
docker exec -it web sh # shell inside a RUNNING container
docker run -it --entrypoint sh myapp # shell in an image that crashes on start
docker build -t myapp:1.0 . # build; "." is the build context
docker inspect web # resolved config after all overrides
docker stats # live CPU / memory
docker system df # disk usage by category
docker system prune # reclaim space
```

Flag	Means
<code>-d</code>	Detached (background)
<code>-p host:container</code>	Publish a port: host first
<code>-it</code>	Interactive + TTY
<code>--rm</code>	Auto-remove on exit
<code>-e KEY=value</code> / <code>--env-file</code>	Environment
<code>-v name:/path</code>	Volume or bind mount



<code>--entrypoint sn</code>	Override the entrypoint, the debugging escape hatch
<code>--restart unless-stopped</code>	Survive crashes and reboots

Lifecycle and exit codes



running `docker run`

PID 1 is alive. `docker ps` shows it.



exited **process ended, or** `docker stop`

SIGTERM, then SIGKILL after ten seconds. Filesystem and logs survive.



gone `docker rm`

Writable layer deleted. Anything not in a volume is lost.

A container lives as long as its main process. That is why `docker run ubuntu` exits at once: its default command is a shell, and a shell with no terminal has nothing to read.

Exit code	Means
0	Finished normally
1	Application error
125	Bad <code>docker run</code> flags
126	Command found but not executable
127	Command not found, often the wrong image
137	Killed (SIGKILL), usually the memory limit



143

Stopped by SIGTERM: a clean shutdown

Dockerfile instructions

Dockerfile

Copy

```
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1
RUN useradd --create-home --uid 10001 appuser
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=appuser:appuser . .
USER 10001
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Instruction	Runs at	Note
FROM	n/a	Always first; pin the tag, never <code>:latest</code>
RUN	Build	Result is saved as a layer
CMD	Run	Default command; replaced by <code>docker run img args</code>
ENTRYPOINT	Run	Fixed executable; <code>CMD</code> supplies its arguments
COPY	Build	From the build context into the image
ADD	Build	Like <code>COPY</code> but unpacks archives; prefer <code>COPY</code>
WORKDIR	Build + run	Directory for everything after it
ENV	Build + run	Persists into the container



ARG	Build Only	visible in <code>docker history</code>
EXPOSE	n/a	Documentation only. <code>-p</code> publishes
USER	Run	Drops from root

Three exam favourites: `RUN` is **build time**, `CMD` is **run time**, `EXPOSE` **publishes nothing**, and `ENV PYTHONUNBUFFERED=1` is why `docker logs` shows Python output without a buffer delay.

Layer caching and COPY order

✓ Dependencies first

- `COPY requirements.txt .` then `RUN pip install`
- `COPY. .` last
- Source edits reuse the cached install
- Rebuild: seconds

✗ Everything first

- `COPY. .` then `RUN pip install`
- Any source change busts the install layer
- Every build reinstalls from scratch
- Rebuild: minutes

Docker reuses a layer when its inputs are unchanged, and **once a layer is invalidated every layer after it is rebuilt too**. Two corollaries to state unprompted:

Layers are additive, so deleting does not shrink or hide. `COPY .env .` then `RUN rm .env` leaves the file in the earlier layer, readable by anyone who can pull the image.

Clean up inside the same RUN . Splitting `apt-get update` and `install` into separate instructions caches a stale package index and ships the package lists.

DOCKERFILE

Copy

```
RUN apt-get update \  
&& apt-get install -y --no-install-recommends curl \  

```



Build context and `.dockerignore`

The trailing `.` in `docker build -t myapp .` names the **build context**: the CLI uploads that whole directory to the daemon before the build starts. Without a `.dockerignore` you ship `.git`, `node_modules`, your virtualenv, and possibly `.env`: slow builds and leaked secrets.

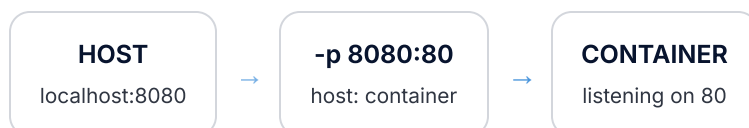
`.dockerignore`

Copy

```
.git
node_modules
__pycache__
*.pyc
.venv
.env
.env.*
dist
build
*.log
```

`docker run --rm myapp ls -la /app` settles "is my file missing?" and "did I ship `.env`?" in one command.

Ports



The mapping is **host first**, and inside a container your app must bind `0.0.0.0`, not `127.0.0.1`, or the published port looks dead. Loopback there means "reachable only from inside this container".



Mechanism	Available at	In the image	Use for
ENV	Build + run	Yes	Non-secret defaults
ARG	Build only	Yes (<code>docker history</code>)	Base versions, build flags
<code>-e</code> / <code>--env-file</code>	Run only	No	Per-environment configuration
Mounted file	Run only	No	Credentials

Precedence, most specific wins: `-e` beats `--env-file` beats `ENV` . **Neither `ARG` nor `ENV` is a secret.** Both are image metadata, readable by anyone who can pull. At this level: secrets arrive at run time.

Volumes

BASH

Copy

```
docker run -v pgdata:/var/lib/postgresql/data postgres:16 # named volume
docker run -v "$(pwd)":/app myapp:dev # bind mount
docker run -v ./conf.yaml:/etc/app/conf.yaml:ro myapp # read-only config
```

	Named volume	Bind mount
Managed by	Docker	You
Location	Docker's storage area	A path on the host
Portable	Yes	No, it depends on a host path
Permissions	Docker initialises them	Keeps the host's ownership



Data you write outside a volume lands in the container's writable layer and dies with `docker rm`.

Networking and Compose

Containers on a **user-defined** network reach each other by container or service name. The **default** bridge has no name resolution. That difference answers most "container cannot reach container" questions.

BASH

Copy

```
docker network create appnet
docker run -d --name db --network appnet postgres:16
docker run -d --name api --network appnet -p 8000:8000 myapp # connects to db:5432
```

compose.yaml

Copy

```
services:
  api:
    build: .
    ports: ['8000:8000']
    environment:
      DATABASE_URL: postgres://postgres:secret@db:5432/app
    depends_on: [db]
    restart: unless-stopped
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes: ['pgdata:/var/lib/postgresql/data']
volumes:
  pgdata:
```



```
docker compose up -d
docker compose logs -f api
docker compose down          # add -v to delete volumes too
```

Compose creates a network and puts every service on it, addressable by service name. That is why the connection string says `db`.

Know the limit of `depends_on`

It waits for the container to **start**, not for the service inside to be **ready**. A database that accepts connections thirty seconds later still breaks your app's first query. The fix: a health check plus `depends_on: { db: { condition: service_healthy } }`, and retry logic in the app regardless.

Non-root, in two lines

DOCKERFILE

Copy

```
RUN useradd --create-home --uid 10001 appuser
COPY --chown=appuser:appuser . .
USER 10001
```

`USER` comes **after** the installs, because installing to system paths needs the privilege you are about to drop. Use a numeric uid. Say the reason out loud: **root in a container is root on the host kernel**, with no hypervisor between them.

Debugging order



02 Exit code

`docker ps -a` . 137 is a kill (usually OOM), 127 command-not-found, 126 not-executable, and 143 a clean SIGTERM.

03 Shell without the app

`docker run -it --entrypoint sh myapp` , then run the command by hand.

04 Is the file there?

`pwd` , `ls -la` inside. A wrong `WORKDIR` or a `.dockerignore` exclusion explains most "not found" errors.

05 Resolved config

`docker inspect NAME` for the real entrypoint, env, mounts, and networks after every override.

The traps, and why they share one cause

Three ideas explain most beginner failures: a container is **one process**, the writable layer is **temporary**, and build time is **not** run time.

Symptom	Cause	Fix
Container exits instantly	The main process finished	<code>-it</code> , or run a real server
Published port refuses connections	App listening on <code>127.0.0.1</code>	Bind <code>0.0.0.0</code>
<code>EXPOSE</code> exposed nothing	It is documentation	Add <code>-p</code>



Build hangs forever	Start command in a <code>RUN</code>	It belongs in <code>CMD</code>
Every build reinstalls dependencies	<code>COPY . .</code> before the install	Manifest first
Data gone after <code>docker rm</code>	It was in the writable layer	A named volume
<code>bash: not found</code>	Slim/Alpine images have no bash	Use <code>sh</code>
<code>docker logs</code> is empty	App writes to a file	Log to stdout
Compose app cannot reach the DB	Used <code>localhost</code>	Use the service name
Deleted secret still in the image	Layers are additive	Rebuild without it, and rotate

Common interview questions

01 What is Docker, and why use it?

It packages an app with its dependencies into an immutable image, so the artifact that passed CI is the artifact that runs in production. It removes environment drift, makes onboarding one command, and gives you fast, disposable, isolated environments.

02 Container versus virtual machine, and is a container less secure?

A container shares the host kernel and is a process with a restricted view: milliseconds to start, tens of megabytes. A VM ships a full guest OS behind a hypervisor: slower, larger, stronger isolation. Yes, weaker: a kernel vulnerability is an escape path, so run non-root and keep untrusted code out of a plain container.

03 Image versus container?



04 Why does `docker run ubuntu` exit immediately?

A container lives as long as its main process. The default command is a shell with no terminal attached, so it finishes at once. `-it` gives it a terminal to wait on.

05 `RUN` versus `CMD` ?

`RUN` executes at build time and its result becomes a layer. `CMD` is the default command when a container starts. Putting the app's start command in `RUN` makes the build hang forever.

06 `CMD` versus `ENTRYPOINT` ?

`ENTRYPOINT` is the fixed executable; `CMD` supplies default arguments. Arguments after `docker run image` replace `CMD` but are appended to `ENTRYPOINT`. Use `ENTRYPOINT` when the container should always run one program, and always use the exec form, so your process is PID 1 and receives `SIGTERM`.

07 Does `EXPOSE` publish a port?

No. It is documentation for humans and tooling. `-p host:container` is what publishes.

08 My port mapping does not work. Why?

Usually the app inside listens on `127.0.0.1`, reachable only inside the container, so bind `0.0.0.0`. Otherwise the `-p` order is reversed, or the app uses a different port than the one you mapped.

09 How do you persist data?

A named volume. Anything you write to the container's writable layer is deleted with the container.

10 Named volume versus bind mount?

A named volume is Docker-managed, portable, and gets correct ownership: right for databases. A bind mount maps a host directory, depends on that path, and keeps host permissions: right for development live-reload, wrong for production data.



name. The default bridge has no DNS, so this fails if you skip creating a network. Compose creates one for you, hence `db:5432` in the connection string.

12 Why is my rebuild so slow?

COPY order, most of the time. Copy the dependency manifest and install first, then copy the source, so source edits reuse the cached install layer. Once a layer is invalidated, everything after it rebuilds.

13 I deleted a secret in a later layer. Is it safe?

No. Layers are additive, so the file is still in the earlier layer and readable by anyone who can pull the image. Rebuild without it and rotate the credential.

14 What is the build context, and why does it matter?

The directory you pass to `docker build`, uploaded in full to the daemon. Without a `.dockerignore` you ship `.git`, `node_modules`, and possibly `.env`: a speed problem and a secret-leak path.

15 How do you pass configuration?

At run time with `-e` or `--env-file`, so you promote one image across environments. Never secrets via `ARG` or `ENV`, because both land in image metadata and `docker history`.

16 How do you debug a container that will not start?

`docker logs` first, then the exit code from `docker ps -a`, then `docker run -it --entrypoint sh` to get inside without running the app, then `docker inspect` for the resolved configuration.

17 What does exit code 137 mean?

The kernel SIGKILLED the process, in most cases at the memory limit. Confirm with `docker inspect --format '{{.State.OOMKilled}}'`.

18 Where should an app write its logs?



19 **Why not run as root, and how do you avoid it?**

Root in the container is root on the host kernel, with no hypervisor between them, so an escape lands as root. Add a user with a fixed numeric uid, `COPY --chown`, and `USER 10001` after the installs.

20 **Why not use `:latest` ?**

It is the tag Docker assumes when you give none, pointing at whatever was pushed last. It can move between two builds an hour apart, so "what is running?" becomes unanswerable and rollback undefined.

Sixty-second self-test

- Give the container-versus-VM answer in two sentences, including the security follow-up.
- Explain image versus container with an analogy.
- Say when `RUN` runs and when `CMD` runs.
- Explain what `EXPOSE` does and does not do.
- Give the two causes of a dead published port.
- Say why `COPY` order changes build speed, and what "additive layers" means.
- Name the storage that survives `docker rm`.
- Explain how Compose services find each other, and what `depends_on` does not do.
- Say what exit codes 137, 127, and 143 mean.
- Give the two lines that make a container run as non-root, and why they go last.





MLOps MENA
COMMUNITY

العربية العربية



[Join the Community ↗](#)



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.